

CYBERSECURITY ASSESSMENT REPORT

OWASP Juice Shop v20 Vulnerability Assessment

Target Application	OWASP Juice Shop v20
Environment	Localhost
Runtime	Node.js v22
GitHub Repository	https://github.com/juice-shop/juice-shop
Prepared By	Ashfaque Ali
Date	12 May 2026

1. Introduction

This report presents the security assessment of OWASP Juice Shop version 20 running on Node.js version 22 in a localhost environment. The purpose of this assessment was to identify vulnerabilities aligned with OWASP Top 10 web application security risks.

2. Objectives

- Identify vulnerabilities in the application
- Analyze security weaknesses
- Understand exploitation techniques
- Map findings to OWASP Top 10
- Provide remediation recommendations

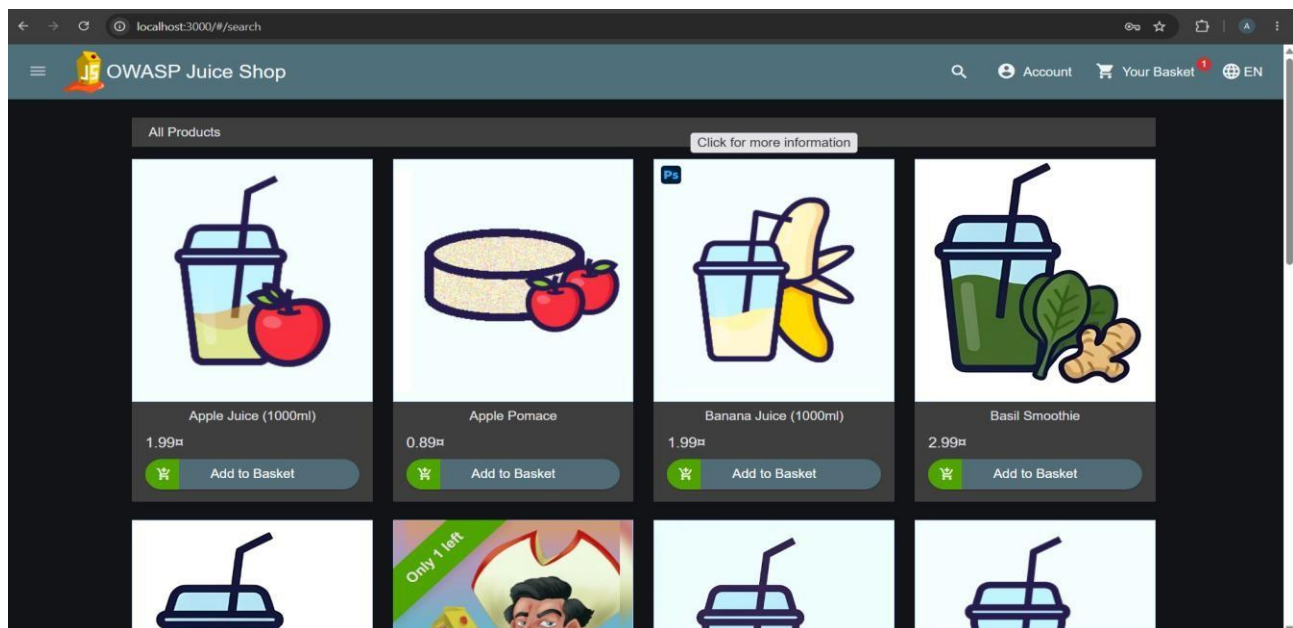
3. Scope of Testing

- Login and Registration
- Search Functionality
- API Endpoints
- Session Management
- Authentication and Authorization
- Client-Side Input Validation

4. Methodology

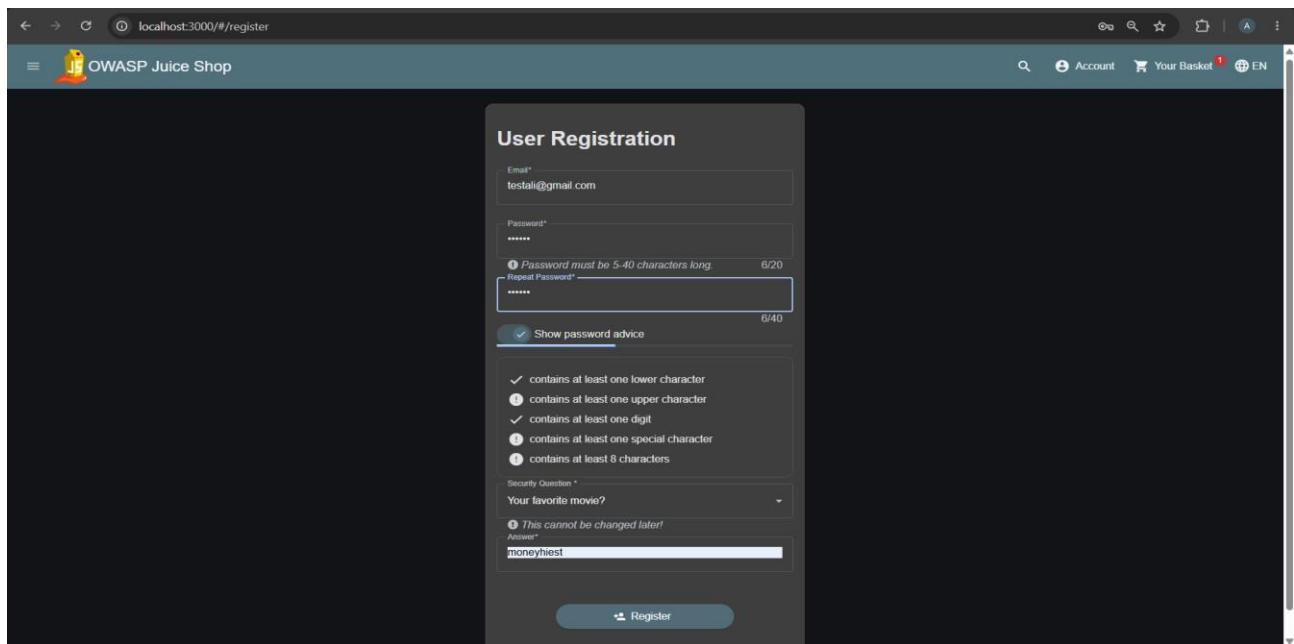
1. Task 1: Reconnaissance

- Identified pages, APIs, and application structure.



Registration:

- Accepts any email format without strict validation.
- Weak passwords like ali123 are accepted without restriction.
- There is not like I am not a Robot.
- Security question answers stored without encryption.

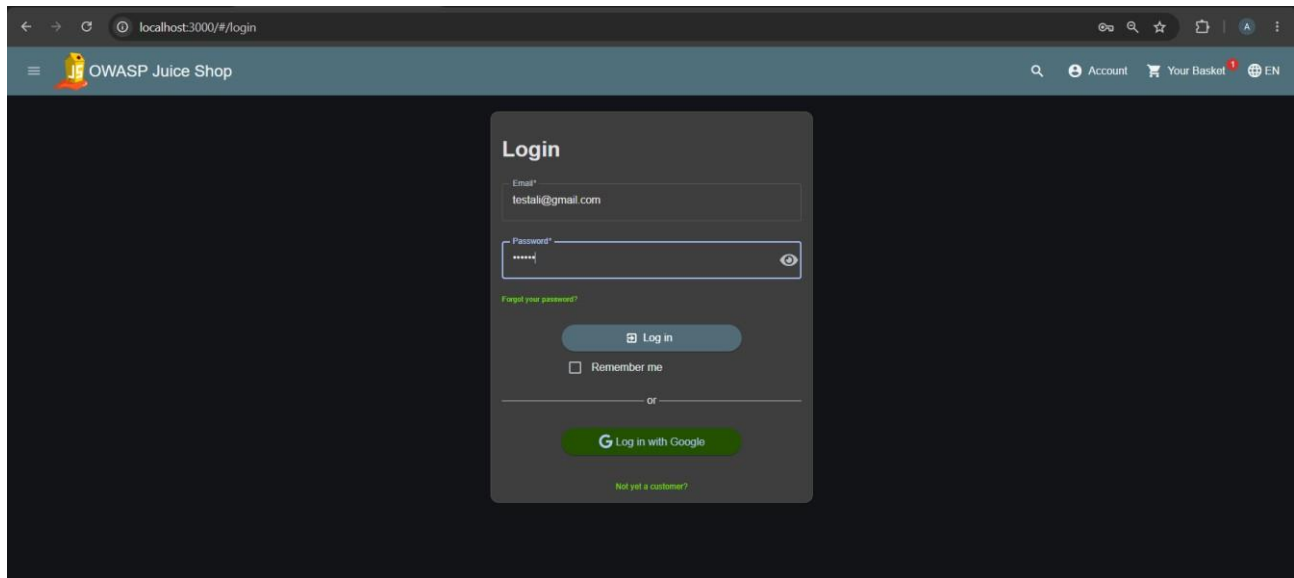


The screenshot shows the 'User Registration' form in the OWASP Juice Shop application. The form is centered on a dark background. It includes the following fields and features:

- Email***: A text input field containing 'testali@gmail.com'.
- Password***: A text input field with masked characters (dots).
- Repeat Password***: A text input field with masked characters (dots).
- Password strength indicator**: A message 'Password must be 5-40 characters long' with a '6/20' character count.
- Show password advice**: A toggle switch that is currently turned on.
- Password requirements list**: A list of five requirements, each with a checkmark icon:
 - contains at least one lower character
 - contains at least one upper character
 - contains at least one digit
 - contains at least one special character
 - contains at least 8 characters
- Security Question ***: A dropdown menu with the selected option 'Your favorite movie?'.
- Answers***: A text input field containing 'moneyhest'.
- Register button**: A blue button with a white arrow icon and the text 'Register'.

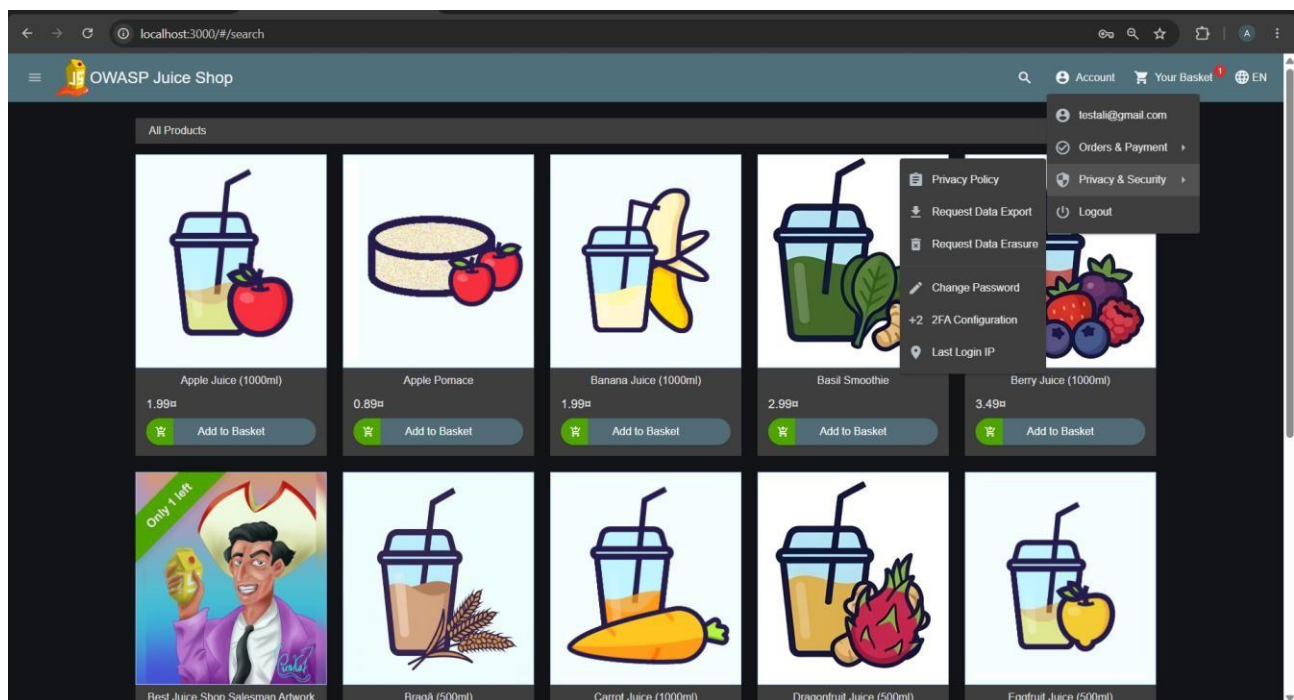
Login:

- No limit on failed login attempts means here is brut force attack can work.
- Error messages reveal whether an email exists in the system.
- Not separate error message for the password.
- There is not the Two-factor authentication (2FA) implemented.
- No account lockout policy after multiple wrong attempts. Like if any user try for three system must lock it.



Account:

- Sensitive user data visible in plain text.
- User data loaded and displayed after authentication.
- Input fields present for updating user information.



2. Task 2: Vulnerability Assessment

- ❑ Performed SQL Injection, XSS, and Authentication testing.

SQL Injection:

The normal database query is:

```
SELECT * FROM users WHERE email='input' AND password='input'
```

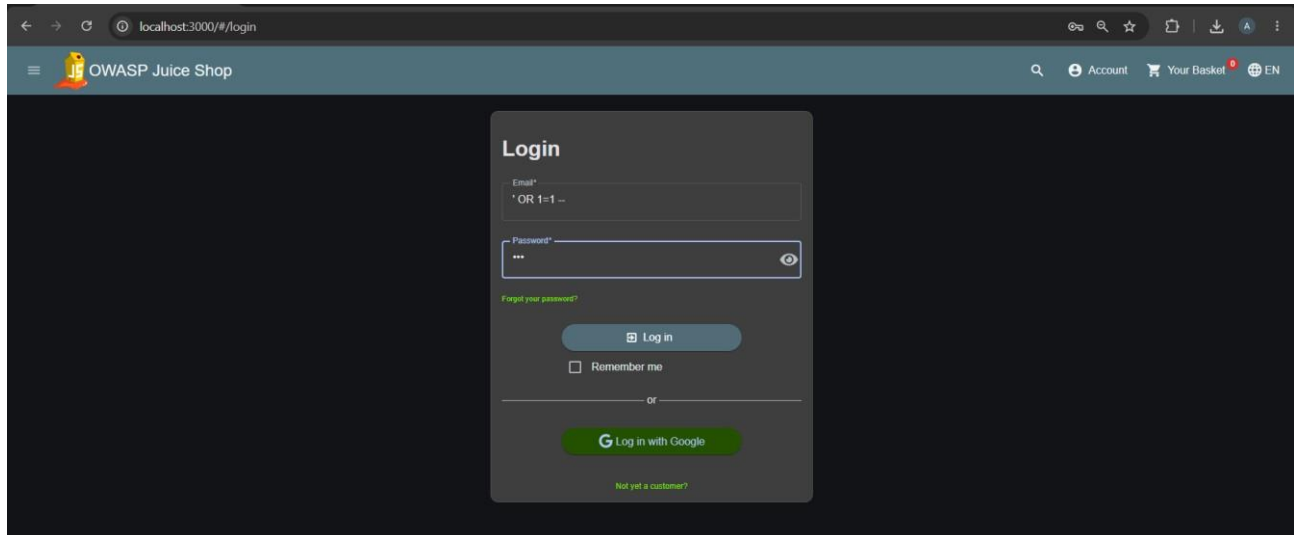
After SQL Injection input, the query becomes:

```
SELECT * FROM users WHERE email='' ` OR 1=1-- AND password='...'
```

Since $1=1$ is always TRUE, and everything after -- is treated as a comment, the authentication is completely bypassed.

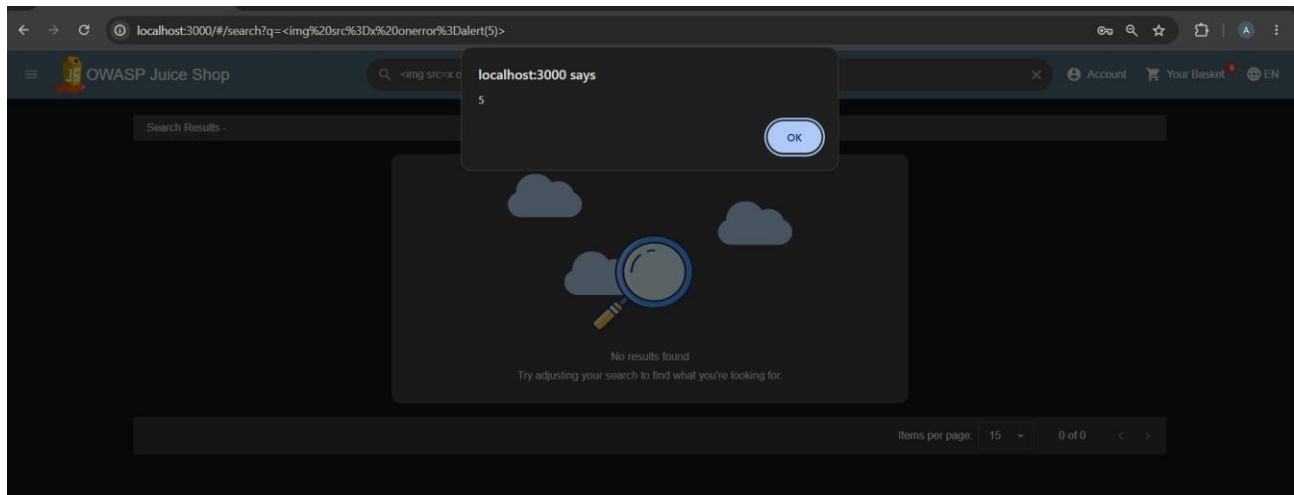
Outputs:

- SQL Injection Successful.
- Admin account accessed without valid credentials.
- Complete authentication bypass achieved.
- The Risk Level is CRITICAL.



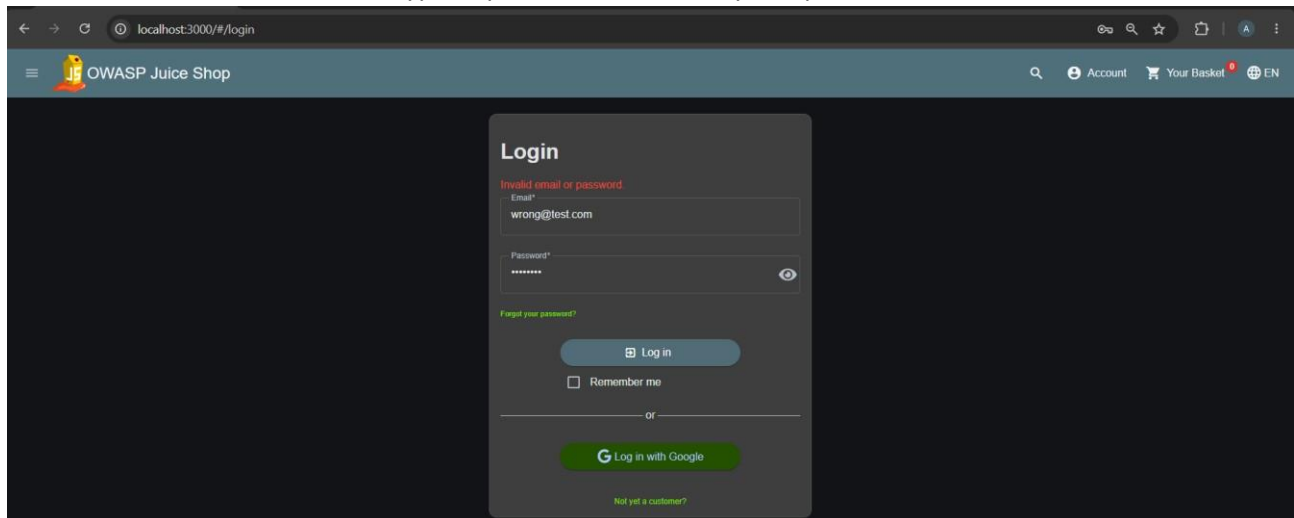
XSS (Cross-Site Scripting):

- User input is safely displayed
- JavaScript `<img src=x onerror=alert(1)>` successfully execute the alert shows.
- By using the search Bar of the application.



Authentication testing:

- Wrong login alert show.
- Weak password registration like 123 it will not accept because they define the parameter like at least password content the 5 characters.
- Logout and login behavior work properly.
- If we use 12345 like this type of password it can easily compromised.



3. Task 3: Documentation:

- Verified findings and collected evidence.

#	Vulnerability	Location	Risk Level	Status
1	SQL Injection	Login Page	CRITICAL	Confirmed
2	Cross-Site Scripting (XSS)	Search Bar / Contact Page	HIGH	Confirmed
3	Weak Password Policy	Signup / Login Page	MEDIUM	Confirmed
4	Sensitive Data Exposure	Browser Local Storage	HIGH	Confirmed
5	No Brute Force Protection	Login Page	HIGH	Identified
6	Missing Security Headers	All Pages	MEDIUM	Identified
7	Verbose Error Messages	Login Page	LOW	Identified

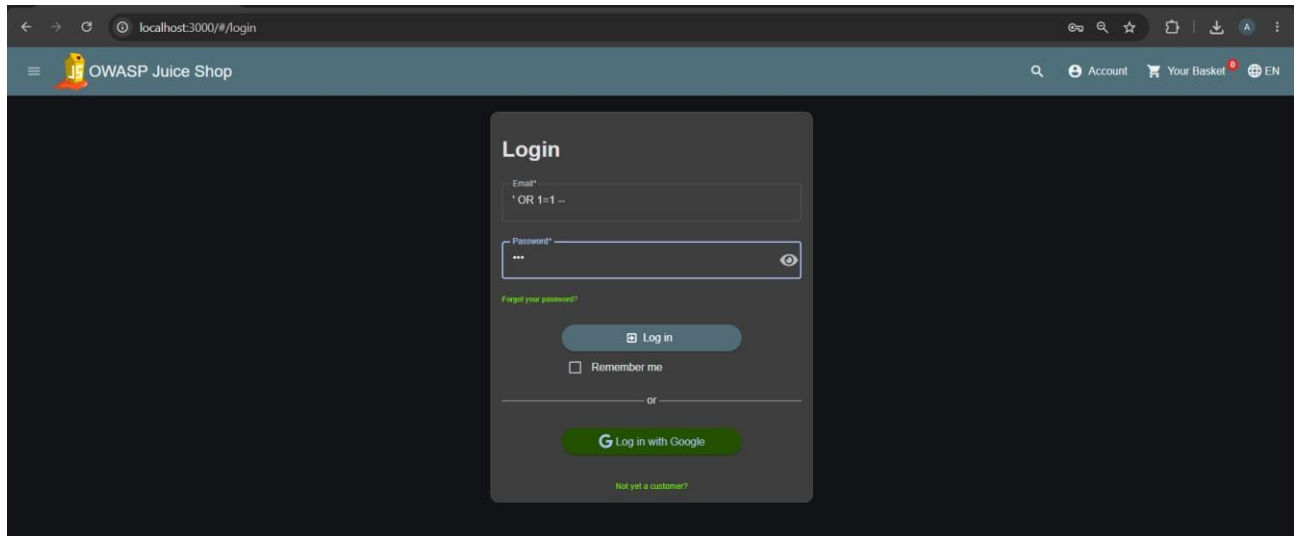
5. Tools Used

- SQL Injection
- OWASP ZAP
- Browser Developer Tools
- Google Chrome
- Node.js v22

6. Vulnerability Findings

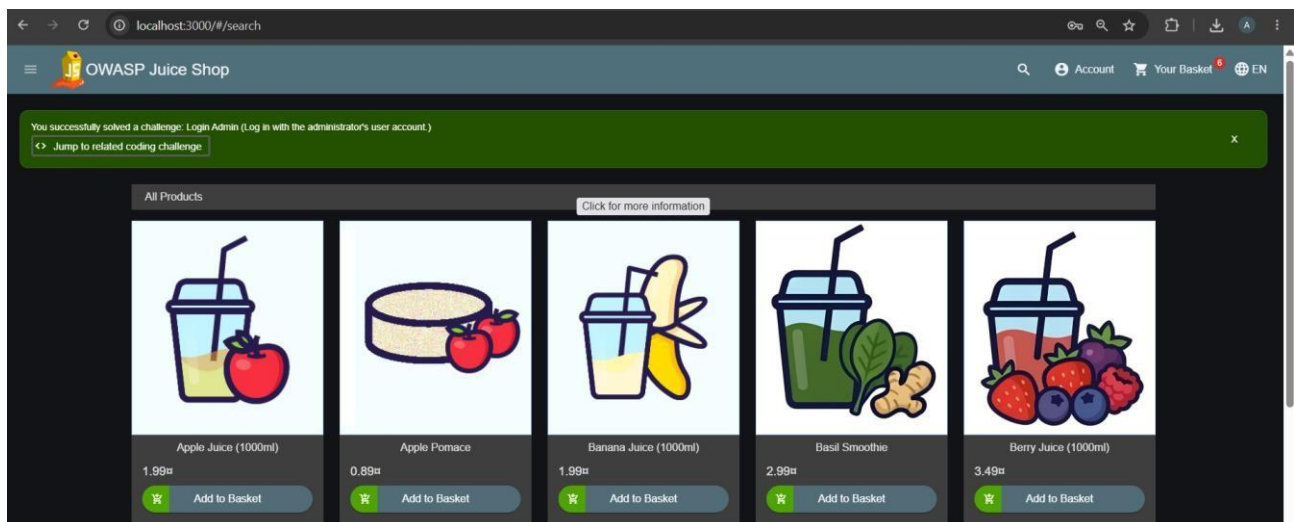
SQL Injection

Severity	Critical
Description	The application was vulnerable to SQL Injection.
Impact	Attackers may exploit this vulnerability to compromise application security.
Recommendation	Apply secure coding and remediation practices.



Cross-Site Scripting (XSS)

Severity	High
Description	The application was vulnerable to Cross-Site Scripting (XSS).
Impact	Attackers may exploit this vulnerability to compromise application security.
Recommendation	Apply secure coding and remediation practices.



Broken Authentication:

Severity	High
Description	The application was vulnerable to Broken

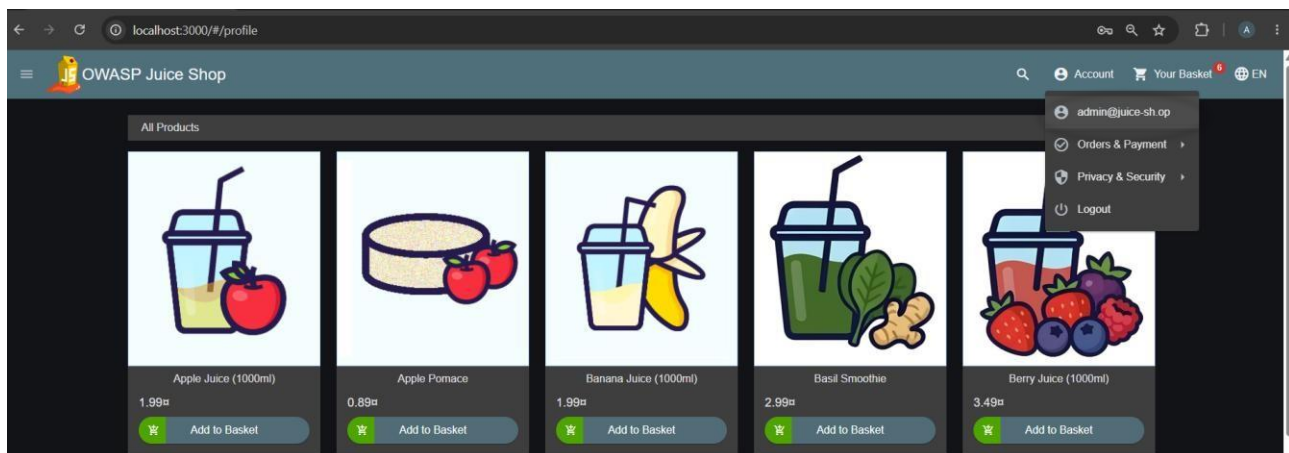
	Authentication.
Impact	Attackers may exploit this vulnerability to compromise application security.
Recommendation	Apply secure coding and remediation practices.

Security Misconfiguration

Severity	Medium
Description	The application was vulnerable to Security Misconfiguration.
Impact	Attackers may exploit this vulnerability to compromise application security.
Recommendation	Apply secure coding and remediation practices.

Broken Access Control

Severity	High
Description	The application was vulnerable to Broken Access Control.
Impact	Attackers may exploit this vulnerability to compromise application security.
Recommendation	Apply secure coding and remediation practices.



7. OWASP Top 10 Mapping

Vulnerability	
SQL Injection	OWASP Category Injection
Cross-Site Scripting	Cross-Site Scripting
Broken Authentication	Authentication Failures
Security Misconfiguration	Security Misconfiguration
Broken Access Control	Broken Access Control

8. Recommendations

- Validate and sanitize all user input.
- Use parameterized queries.
- Apply strong authentication mechanisms.
- Configure secure HTTP headers.
- Conduct regular security testing.
- Update application dependencies regularly.

9. Conclusion

The assessment identified multiple vulnerabilities aligned with the OWASP Top 10 categories. The findings demonstrate the importance of secure coding, proper authentication, and regular security testing.